

REPIF1: SOFRA Jonathan

Indoor Climate Station



Indoor Weather Station

Contents

Installation and Configuration of RaspberryPI	3
Installation of the Program	3
Using the official Imager	3
Hardware Part.....	5
Touching the Touchscreen	6
Starting the RaspberryPI with the touchscreen.....	7
Setting up VNC Server for remote desktop	8
Setting up a static IP address for the Ethernet Interface	10
Enabling I2C Interface	11
Installing Docker and PHP with Apache2.....	11
Installing Firmware Files and configuring them	12
Autostart firmware	14

Installation and Configuration of RaspberryPI

On the RaspberryPI official's website, a tool called RaspBerryPI Imager can be installed to flash the RaspberryPI OS on a SD card that we would later on use for the RaspberryPI itself.

We will later on also configure the hardware, meaning mounting also the touchscreen.

Installation of the Program

Head to the website: <https://www.raspberrypi.com/software> and download **Raspberry Pi Imager**.

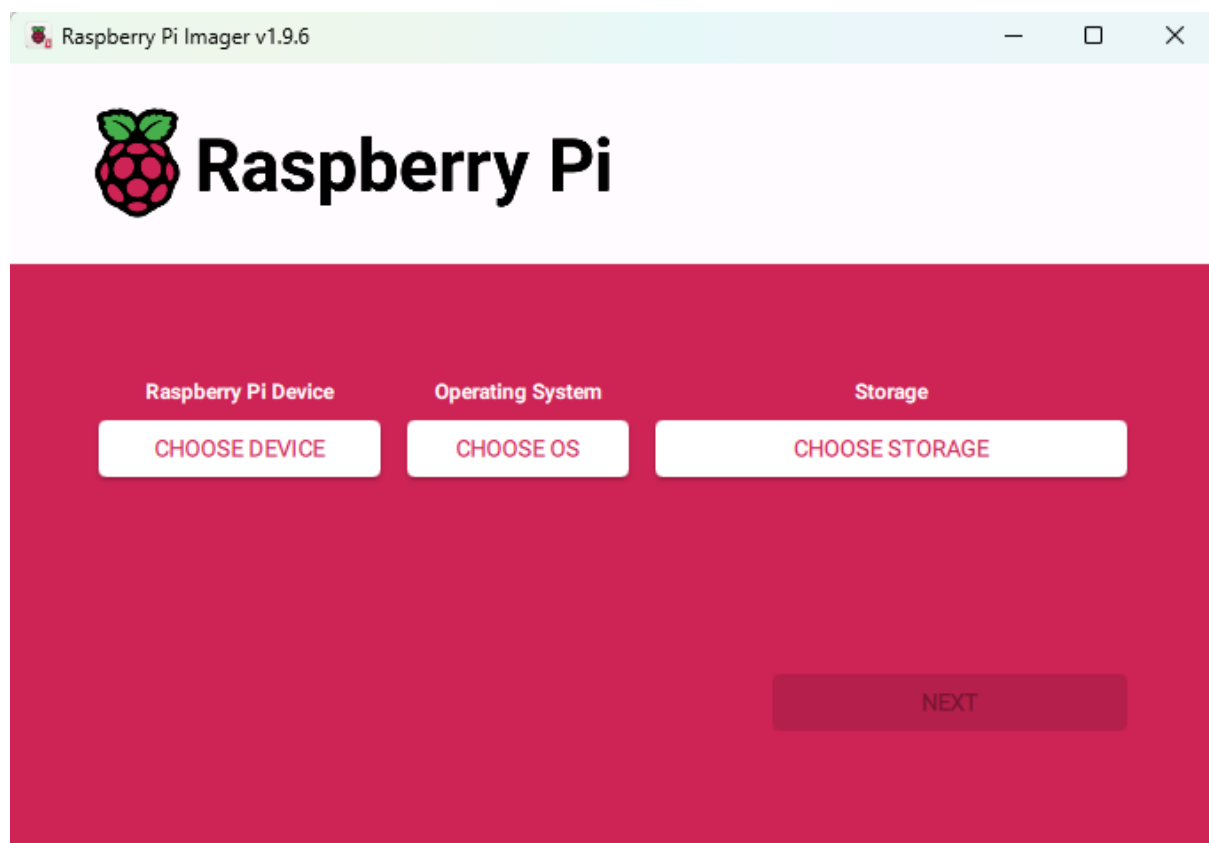
This involves us to do a normal and easy setup I will document.

The first step of the setup is just a basic language selector, in my case I will select **English**.

The first fews steps, there is nothing special to do except clicking **Next** until it's successfully downloader on our system.

Using the official Imager

Now after the installation, you should have the same program right here:



It is pretty self explanatory, select the correct Device, Operating System and Storage.

In our case we will select **RaspberryPi 4** as the Device, the **RaspberryPi OS 64-bit** and the storage, our SD card mounted on a usb sd-card reader/writer.

After clicking **Next**, we have to **Edit Settings** of the OS customization, as we have to set up some important details.

We will set-up the **hostname** so that we can later on SSH into it without IP by using the local domain, we will also setup the **username** and **password**, the **wireless LAN** as the project should be wireless, and optionally the keyboard layout and timezone so that we don't have to deal with it after the installation of the OS. And most importantly we will also enable **SSH** service.

Here is my configuration:

The screenshot shows the 'General' tab of the Raspberry Pi OS Customisation window. The following settings are visible:

- Set hostname:** .local
- Set username and password:** ☒ (checked)
 - Username:
 - Password:
- Configure wireless LAN:** ☒ (checked)
 - SSID:
 - Password:
 - ☐ Hidden SSID
 - Wireless LAN country:
- Set locale settings:** ☒ (checked)
 - Time zone:
 - Keyboard layout:

Buttons:

The screenshot shows the 'Services' tab of the Raspberry Pi OS Customisation window. The following settings are visible:

- Enable SSH:** ☒ (checked)
- Use password authentication:** ☒ (selected)
- Allow public-key authentication only:** ☐ (not selected)
- Set authorized_keys for 'sofjo685':**
- Buttons:

After editing the settings we click **Save** and we apply the **OS customisation settings** by clicking **YES**.

If it prompts us to click on **YES** to erase storage, do it.

After a small wait, the installation should be finished, and we will pursue on the configuration and mounting of the hardware.

Hardware Part

After the OS is installed into our SD card we will start by the hardware part, meaning unboxing of the necessary components, meaning:

- **The Power Supply**
- **The RaspberryPI Model 4B 1GB RAM**
- **The Touchscreen**
- **A screwdriver**

Here is a photo of all the components necessary for the mount of the touchscreen

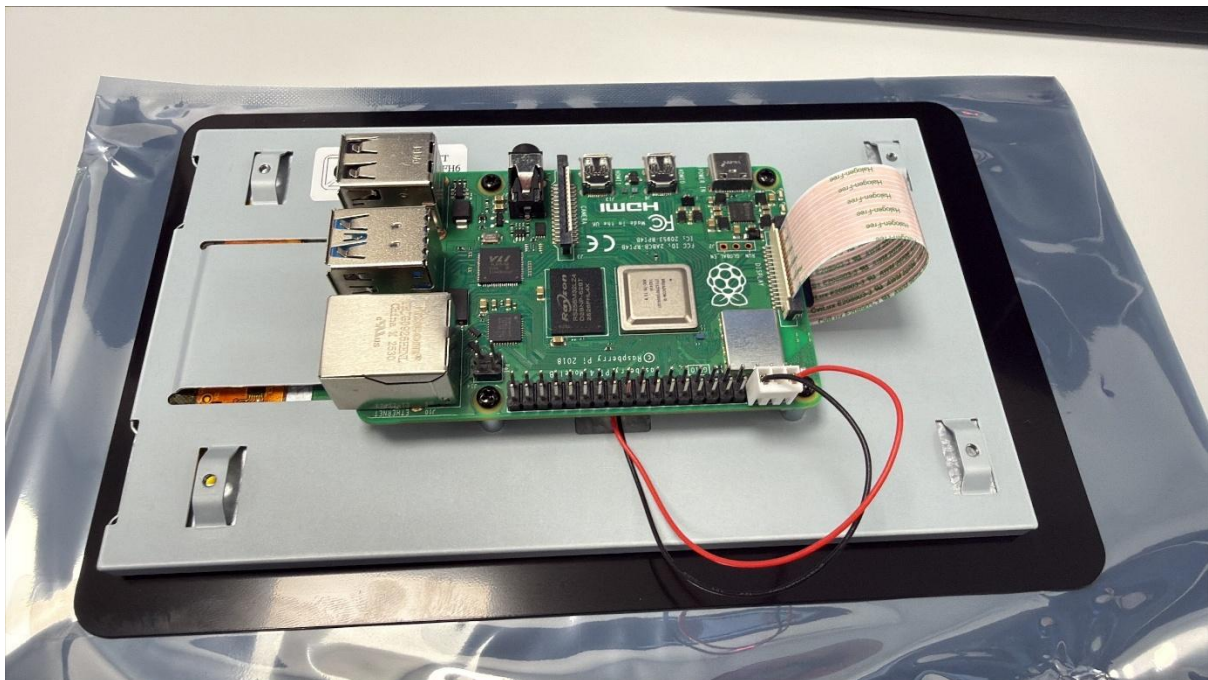


Touching the Touchscreen

We will now start connecting the touchscreen. Here is an image from the official documentation showing how it should be done.



And here is mine done.



Starting the RaspberryPi with the touchscreen

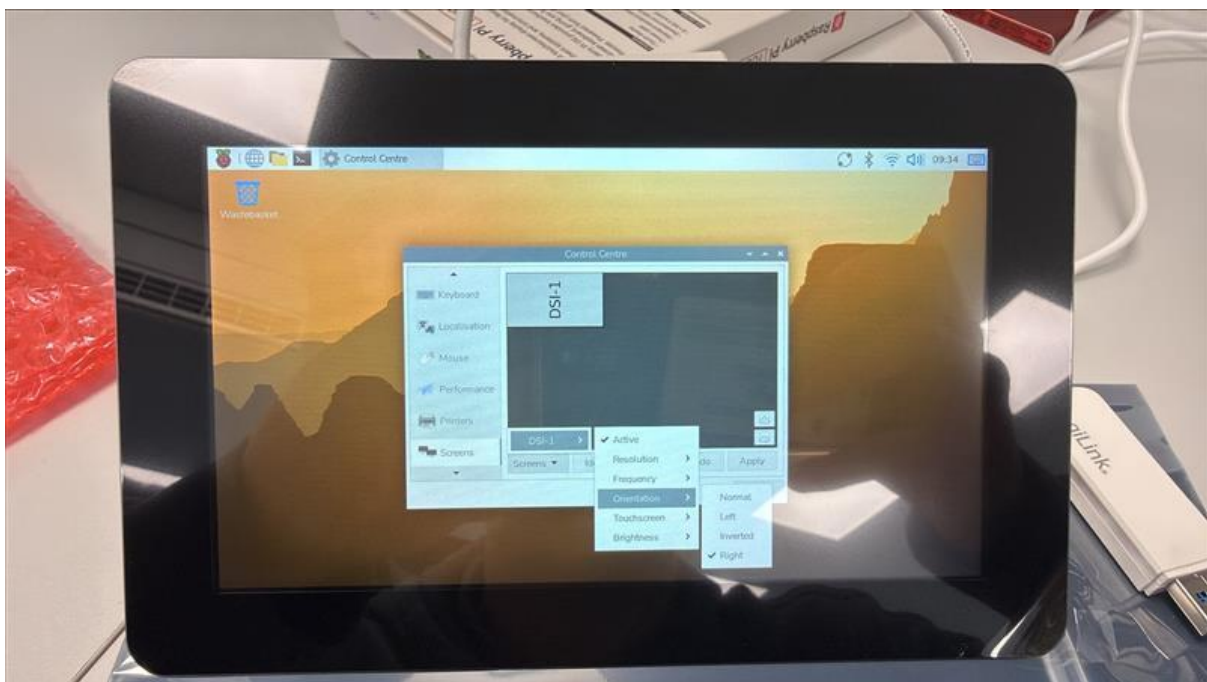
I will now plug the RaspberryPi using the **Power Supply** cable.

After the booting of the OS, we see that the Display mode is set as default as **Portrait**.

We will immediately change it as **LandScape**.

In the RaspberryPi Control Center, head up to the **Screens** section, then click on **Screens -> DSI-1 -> Orientation -> Right**.

It should now be displayed as **LandScape**.



Note: You might need to reboot the OS to ensure that everything is displayed correctly.
(We will also do an update using `sudo apt update && sudo apt upgrade`)

Setting up VNC Server for remote desktop

For this step, we will use ssh to connect remotely to the RaspberryPi.

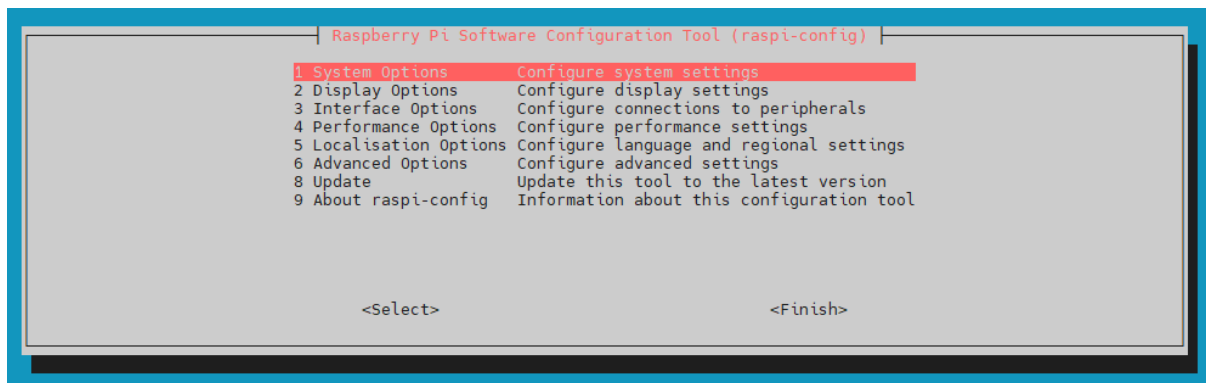
I personally recommend MobaXTerm as it is really user friendly and easy to use.

The IP is always shown when raspberry just got connected to the network.

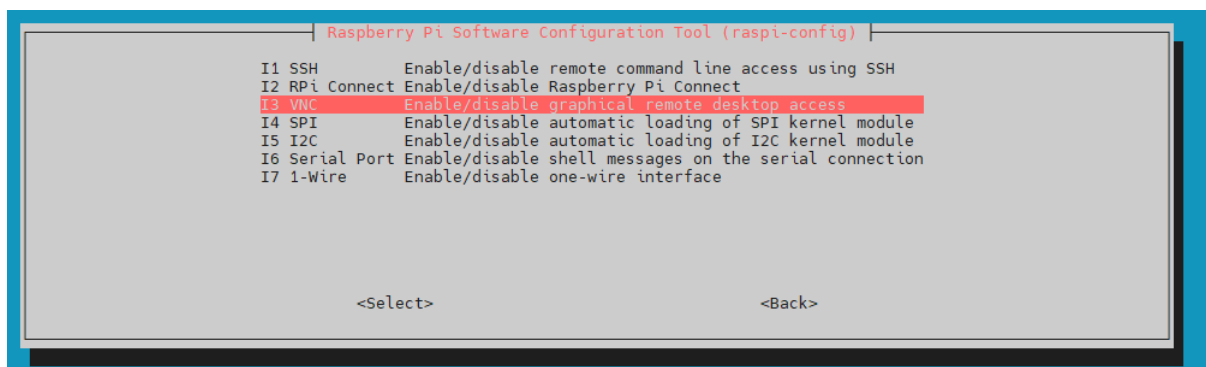
The command we are going to use is :

```
sudo raspi-config
```

And you should see this



Then you should active VNC by going through the 3rd row named “Interface Options” then press ENTER on VNC to enable VNC Server.

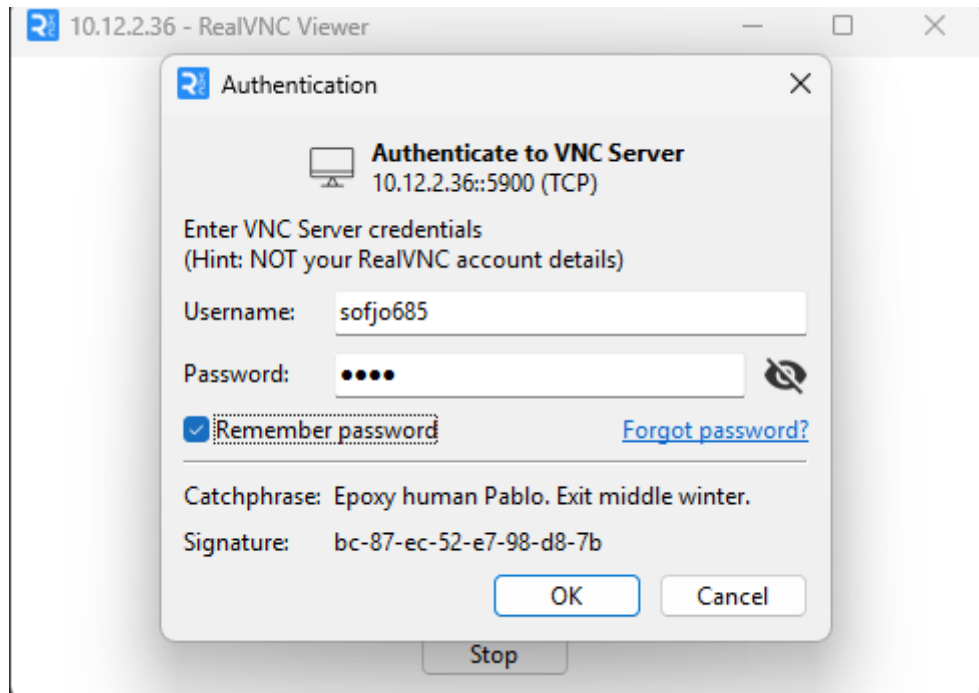


Connecting to our RaspberryPI through VNC

For this step we will use **RealVNC Viewer**.

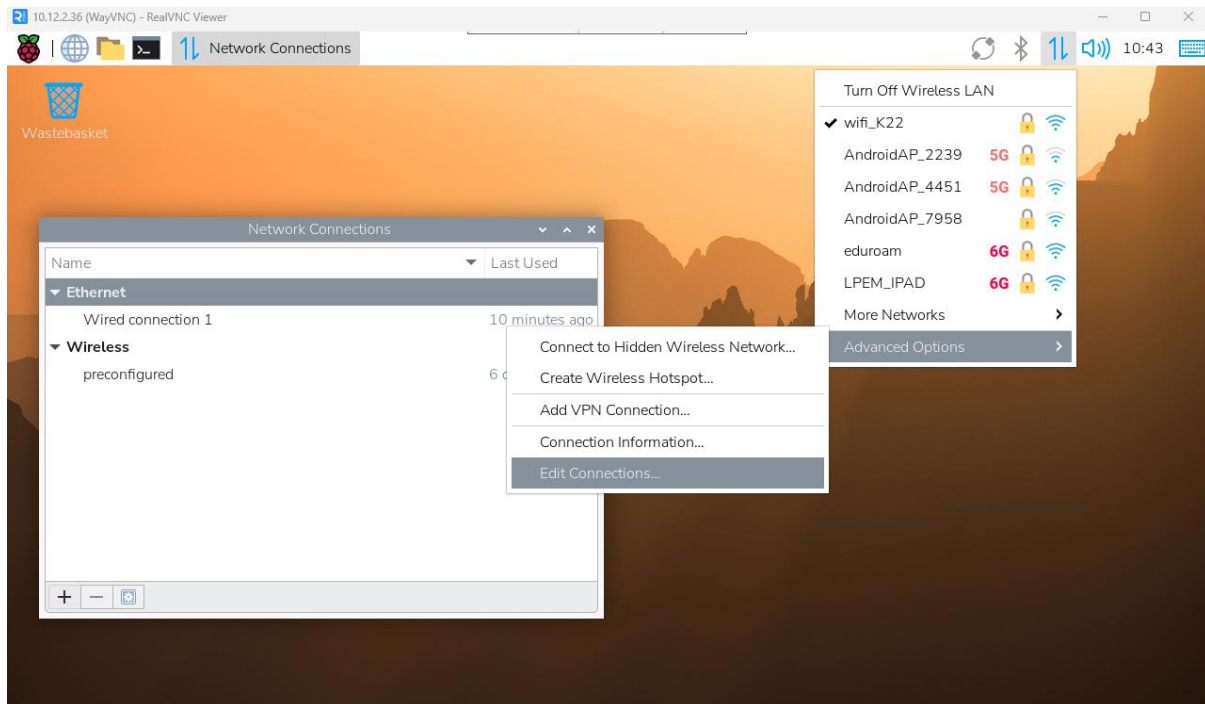
It is really user friendly, you should just put the IP address of your RaspberryPI in the searchbar then press ENTER.

Then connect as normal to your user.



Setting up a static IP address for the Ethernet Interface

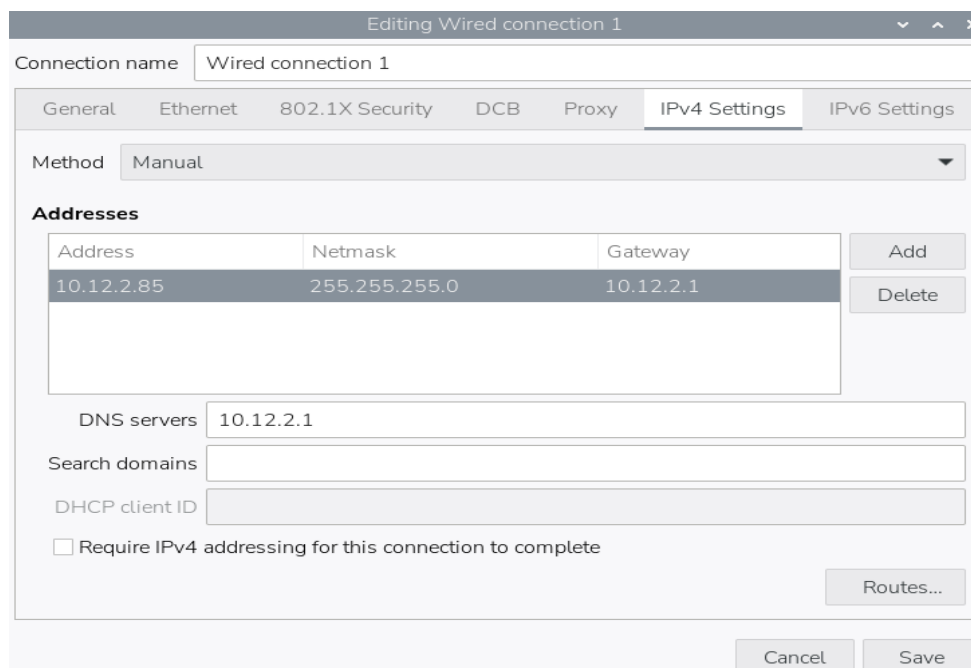
To set up a static IP, click on the network icon on the top task bar top-right, then click on **Advanced Options -> Edit Connections...**



You will see your network interfaces, wireless and ethernet.

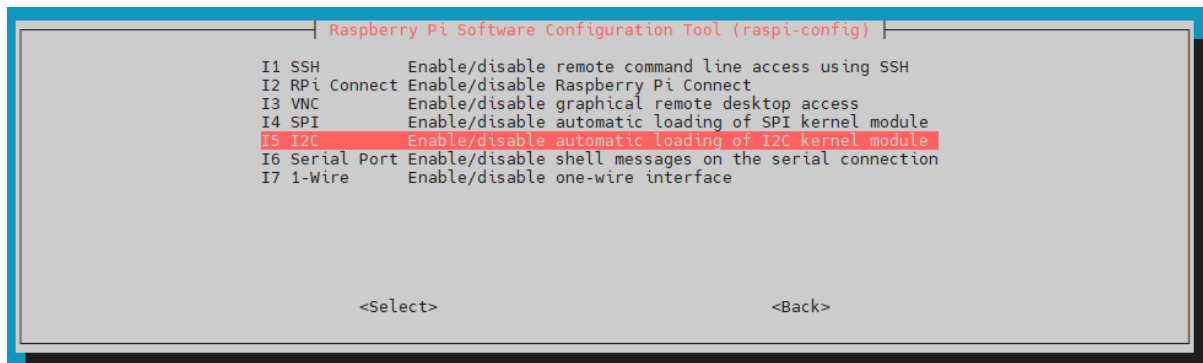
Click on **Wired connection 1** then click the **Option** icon bottom-left.

Go to **IPv4 Settings** and change method to **Manual** and set the correct IP, I will use the last 2 digits of my IAM Code.



Enabling I2C Interface

As we did to enable VNC Server, we will re-use the **sudo raspi-config** command to enable the I2C Interface, it is also located in the same section as **VNC**.



Installing Docker and PHP with Apache2

Quick install Docker using installation script

```
curl -sSL https://get.docker.com | sh
```

This installer will install everything on its own.

After the installation is done, we will use a docker container to run php with apache2.

```
sudo docker run -d --name php -p 8080:80 -v /tmp/station:/tmp/station -v  
/var/www/html:/var/www/html php:8.4-apache
```

Install also the Python dependencies

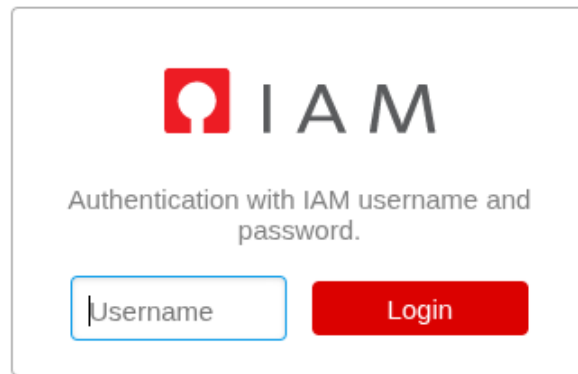
```
sudo apt install python3-smbus2 python3-venv python3-requests  
  
cd /opt/station  
sudo python3 -m venv venv  
source venv/bin/activate  
pip install bh1750 bme280-sensor mq135-sensor
```

Installing Firmware Files and configuring them

In this project, we need some firmware files to ensure the correct functionality of our station.

The URL is the following one: [Ressourcen.zip](#)

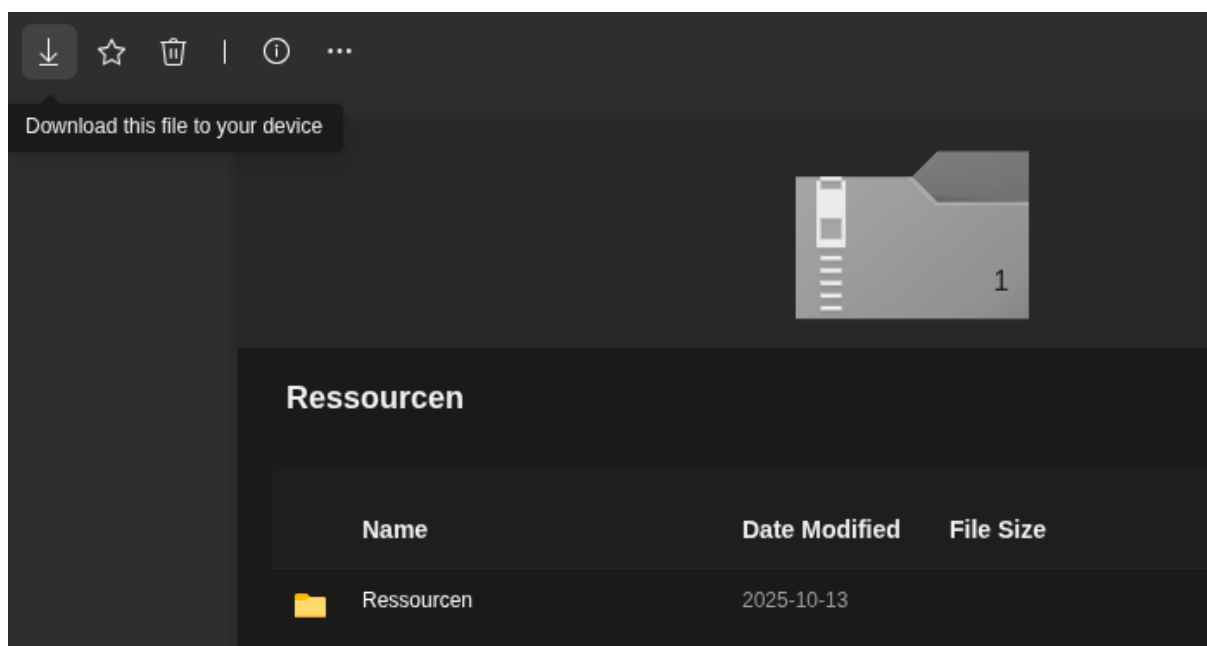
You shall connect to your IAM to gain access to the download

A login form for IAM. It features a red square icon with a white circle inside, followed by the text 'IAM'. Below this, it says 'Authentication with IAM username and password.' There is a text input field labeled 'Username' and a red 'Login' button.

 IAM

Authentication with IAM username and password.

After logging in, you should download the ZIP file

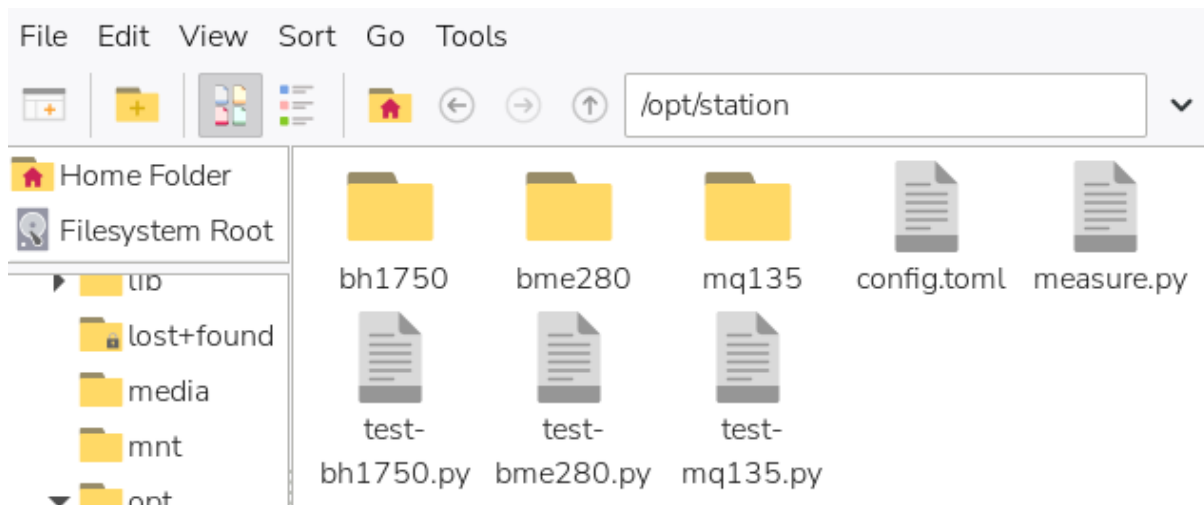


Now we will set them in the correct folders with the correct permissions.

Here are the commands:

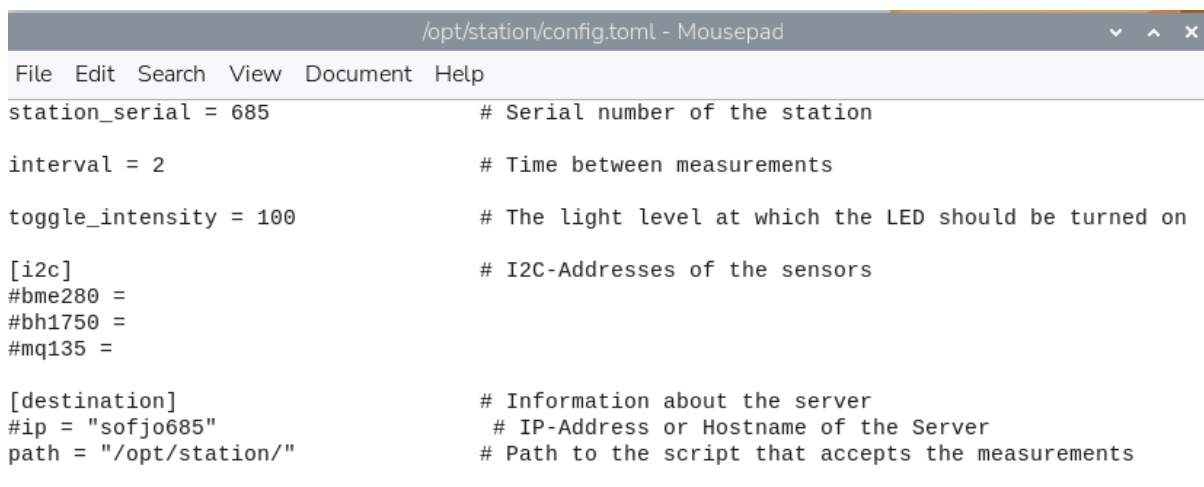
```
sudo mkdir -p /opt/station/www
sudo mkdir -p /var/station
sudo mkdir -p /tmp/station
sudo mkdir -p /var/www/html
sudo chown -R sofjo685:sofjo685 /opt/station /var/station /tmp/station
/var/www/html
```

In the ZIP file, we will extract all files from **Ressourcen/Firmware und Scripts/** to **/opt/station**



We will now copy the index.php file from the ZIP archive to **/var/www/html** as the Docker container mounts **/var/www/html** directly.

Edit **config.toml** file, for now we will only change the **station_serial** to a unique number and the **path** to **/opt/station**



Autostart firmware

Modify or create autostart file using command

```
nano ~/.config/labwc/autostart
```

You will put this code, I commented each section to explain what they do.

```
#!/bin/bash
# Create necessary directories
mkdir -p /tmp/station

# Start Docker container with website from /var/www/html
sudo docker start php || docker run -d --name php -p 8080:80 -v
/tmp/station:/tmp/station -v /var/www/html:/var/www/html php:8.4-apache

# Wait for services to start
sleep 5

# Start measurement script using virtual environment
cd /opt/station
source venv/bin/activate
python3 measure.py &

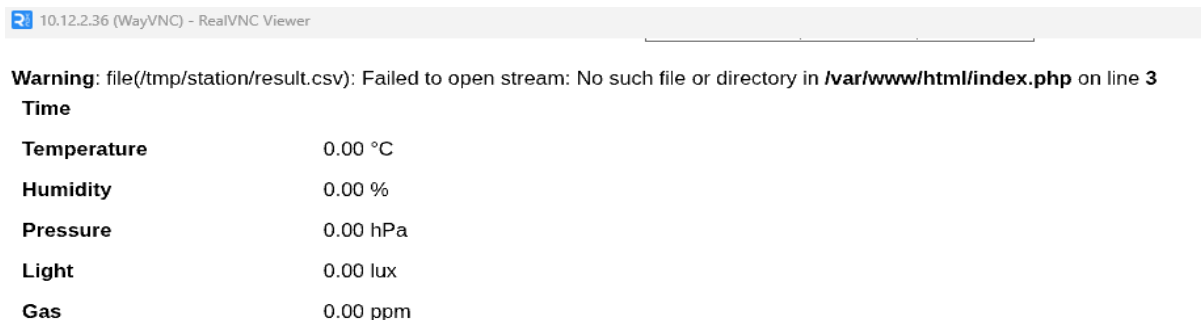
# Wait for measurements to initialize
sleep 10

# Start browser in kiosk mode (fullscreen)
chromium --kiosk http://localhost:8080 &
```

Now make sure that the file has the correct permission to execute

```
sudo chmod +x ~/.config/labwc/autostart
```

After rebooting, you should now see the web interface in fullscreen running like this:



Note: If you see any errors in the beginning, just ignore them.